

Mini Project 14 — Pharmacovigilance Adverse Event Intake and Triage Service

Course: UE24AM342AA5 — ML System Design and AgentOps | **Domain:** Pharmaceuticals and Drug Safety
Component: Jackfruit (Level 3), 10 marks | **Team size:** 2–3 students

Technical terms used here are defined in 00_GLOSSARY.pdf.

1. Context

Pharmaceutical companies operate a pharmacovigilance function. Reports of adverse events arrive from doctors, pharmacists, patients and published literature. **Each one must be assessed and, depending on the assessment, reported to the regulator within a fixed number of days.**

Two assessments come first. **Seriousness** — did it cause death, hospitalisation, disability or a congenital anomaly. **Expectedness** — is this reaction already listed in the approved product information.

The reporting timeline depends on the combination, not on either assessment alone. Some combinations require expedited reporting within a fixed number of days; others are reported periodically. **Missing an expedited timeline is a regulatory finding.**

2. What to Build

A multi-agent service that accepts an adverse event report, validates it, assesses seriousness and expectedness against the reference documents, and routes it to the correct reporting stream with a cited justification.

This is a production service, not a notebook.

The system never submits a regulatory report. It produces an assessment with supporting evidence. A qualified person for pharmacovigilance decides.

3. Corpus

CDSO publishes the Rules, the Indian Pharmacopoeia Commission publishes its guidance, and WHO-UMC publishes the causality criteria. All free. Package inserts are published by manufacturers. All documents below are free to download.

- The New Drugs and Clinical Trials Rules, 2019, the provisions on adverse event reporting
- The Indian Pharmacopoeia Commission pharmacovigilance guidance for marketing authorisation holders
- One approved package insert for a marketed drug

Prefer documents with a text layer. A scanned document needs optical character recognition (OCR) before it can be indexed.

Why this corpus is difficult. Reporters describe reactions in everyday words. Package inserts list them in medical terminology. A search using the reporter's words will often miss the listed reaction.

4. What the System Must Do

#	Requirement
F1	Accept an adverse event report over a documented REST interface, against a published data contract
F2	Enforce schema, range, categorical and cross-field validation at the ingestion boundary . Quarantine rejects.
F3	Extract the suspect drug, reaction, demographics, temporal sequence and outcome entities the assessments need
F4	Retrieve the listed reactions, the seriousness criteria and the reporting provision that applies
F5	Produce a seriousness and expectedness determination grounded in retrieved text , with source citations
F6	Route to one of: expedited report, periodic safety report, or non-reportable with reason. State the basis.

#	Requirement
F7	Escalate to a human instead of answering when the reaction cannot be matched to the package insert with confidence, the timing is unclear, or the drug falls outside your corpus
F8	Expose health, metrics and structured logs for the whole pipeline

F5 and F7 hold most of the difficulty. A system that always answers is easy to build; one that knows when not to answer is not.

4.1 The difficult case

The system will be tested against this scenario at the demonstration.

"The patient's skin started peeling badly a week after starting the drug, and she was admitted to hospital."

The reporter's words will not match the package insert, which lists reactions in medical terms. The system must establish whether the reaction is listed, apply the seriousness criteria, and route the report under the reporting provisions, citing each. Adapt the reaction described to the drug you select.

At Evaluation 1, show the passages in your corpus that this case depends on, and the values you will test with. Once confirmed, this is the reference answer for your demonstration.

5. Design Approach

The straightforward implementation embeds the request, retrieves the highest-scoring passages, and asks the model to answer from them. This handles ordinary cases but not the case in section 4.1.

The reason is structural. A single retrieval pass returns passages resembling the question asked. The case in section 4.1 turns on a provision the request never mentions, so a single embedding-based search is unlikely to return it.

The required approach is an agent that:

1. Retrieves the provision that appears to apply
2. Determines whether any other provision modifies it, and searches for such provisions
3. Searches again, using a query derived from what it has read
4. Verifies that the retrieved passages support the answer
5. Retries with a different strategy, or escalates, if they do not

Steps 2 and 4 distinguish an agent from a fixed pipeline. Both are assessed.

6. Agent System Requirements

Unit 4 — agent architecture, orchestration, containerisation, observability.

#	Requirement
A1	Build the reasoning layer with LangGraph or CrewAI . Retrieval must be a tool the agent chooses to call , not a fixed stage. Provide at least three tools, such as corpus search, lookup of a clause by identifier, and lookup of the request's own fields. Bound the loop by steps, tool calls and wall-clock time.
A2	At least two agents with distinct responsibilities and a defined hand-off. Document the topology and what happens if one fails mid-run. Two prompts in one process are not two agents.
A3	Verify every answer against the passages retrieved, before returning it. Every citation must resolve to retrieved text. If verification fails, search again with a different strategy or escalate. An unverified answer must not be returned. This is the basis of F7.
A4	Expose the service with FastAPI . Runs take tens of seconds, so execute them asynchronously : return a run identifier immediately, and the caller polls a status endpoint or receives a callback. Justify the choice. Place a queue or task runner between submission and execution.
A5	Timeouts on every model and tool call, bounded retry with exponential backoff , a circuit breaker on the provider, and graceful degradation along a defined fallback path. All must be demonstrable on request. A run that did not complete must not be returned as a confident answer.

#	Requirement
A6	Track per agent and display on the dashboard: task success rate , steps and tool calls per run, latency end to end and per step, retry rate , container uptime.
A7	<code>docker compose up --scale <agent-worker>=3</code> must work. Measure the effect on throughput and p95, using the stub described in D6. If scaling does not help, identify the actual bottleneck.
A8	A simple web client that submits a request, obtains the result, and displays the recommendation with citations and the escalation path. No marks for appearance.

7. LLMOps Requirements

Unit 3 — *retrieval, prompt management, evaluation, guardrails, monitoring.*

#	Requirement
L1	Orchestrate retrieval with LangChain or LlamaIndex ; index with FAISS or Chroma . Document the chunking and embedding strategy and how it addresses the difficulty in section 3. Re-embedding must be possible without rebuilding the whole index.
L2	Treat prompts as versioned artifacts , and record the prompt version with every response. Compare at least two prompt variants on the same evaluation set; report which performed better, on which metric, and by how much.
L3	Build a fixed evaluation set of at least 20 scenarios , each with its expected outcome and the passage that supports it, including the case in section 4.1, runnable as a batch from the command line. Use TruLens or equivalent. Report groundedness, context relevance, faithfulness and hallucination rate , stating how each is computed. Then make one deliberate change and report all four metrics before and after .
L4	Prompt injection defence on every free-text field reaching a model, output filtering so the system cannot issue a determination it is not authorised to make, and toxicity detection on input and output. Agents are more exposed than pipelines, because injected text can reach a model holding tools.
L5	Record prompt tokens, completion tokens and cost at the time of the call , attributed per step as well as per run . Report cost per 100 runs and the largest single cost driver.
L6	A feedback endpoint allowing the caller to mark a result satisfactory or not, with a reason, persisted and aggregated. Score a small sample of live runs automatically against the evaluation metrics, charting quality over time.

8. Platform and Deployment Requirements

Units 1 and 2 — *data contracts, architecture, containerisation, observability, reliability.*

#	Requirement
D1	Publish a data contract for every external interface. Enforce schema, type, range, category and cross-field validation at the ingestion boundary , before any agent starts. Quarantine rejected records with their field-level errors . A malformed request must not start an agent run.
D2	An idempotent pipeline that reads, parses, chunks, embeds and indexes the corpus, invocable from a command-line tool. Record data freshness — when the index was built, from which versions, how many chunks — and the pipeline's throughput, and expose both as metrics.
D3	At least four services under Docker Compose, starting with a single <code>docker compose up</code> from a clean clone, with dependencies and health checks declared. Nothing runs outside the containers. Justify the placement of deployment boundaries on scaling, failure isolation, independent release or resource profile.
D4	No credentials, endpoints or model identifiers anywhere in the repository . All environment-specific configuration injected at runtime, with a documented <code>.env.example</code> .
D5	Structured logging throughout, a correlation ID across every service and agent step, tracing with OpenTelemetry exported to Jaeger or Grafana Tempo, Prometheus scraping every service, and a Grafana dashboard showing p50/p95 latency, error rate, throughput, availability and resource utilisation per container. Given a run identifier, the entire run must be reconstructible — every tool call, query, passage, prompt version, per-step latency and cost, and the verification outcome.

#	Requirement
D6	Declare a p95 service level objective (SLO) for a complete run before measuring anything , justified against section 1. Measure a run from submission to final result, not the submission call alone. Load test with Locust , with the LLM replaced by a stub service that returns fixed responses after a realistic delay, so that the test measures your system rather than the provider's rate limits. Report RPS, p50 and p95 against the target and state whether it was met. Identify the concurrency at which the target fails and the component responsible . A target set after seeing the results does not satisfy this requirement.
D7	Caching — of embeddings, retrievals or whole runs, justified — and rate limiting per client. Report the measured effect of caching on p95 and cost per run. Rate limiting is a cost control here: one client looping against a multi-step agent can exhaust the budget in minutes.
D8	A runbook for the two most likely failure modes — symptom, confirmation, recovery — and an incident log with at least one genuine entry recording a real failure during development.

9. Demonstration

Prepare all ten items.

#	Item
1	docker compose up from a clean clone to a working system
2	A valid request producing a routed recommendation with citations that resolve to retrieved text
3	One run traced end to end — every tool call, query, passage, prompt version, per-step latency and cost
4	A malformed request rejected and quarantined, with evidence that no agent run started
5	A case the corpus cannot answer producing an escalation rather than a fabricated response
6	Self-verification catching an unsupported answer, followed by a further search or escalation
7	A prompt injection attempt and the guardrail response
8	The circuit breaker opening when the provider is unavailable, and the degradation path
9	Locust results against the declared target, the concurrency at which it fails, and the effect of --scale
10	The case in section 4.1, handled correctly

10. Assessment

If the system does not start and serve a valid request at the demonstration, the project is awarded zero marks, irrespective of the report, presentation or repository.

Establish a working end-to-end path by week two and add requirements after that. Completing all 22 requirements is not expected; sound prioritisation is itself assessed.

Criterion	Marks
Problem understanding, architecture and objective clarity	10
Presentation, teamwork and communication	10
Engineering judgement — tool selection, scope, handling of constraints	10
Report, code quality and repository documentation	10
Total (Evaluation 2, scaled to 8 marks)	40

11. Deliverables

Deliverable	Due
Corpus confirmation and project plan	Evaluation 1
Report and final GitHub repository	One week before the presentation
Live demonstration and viva	Evaluation 2

Report — a plain report covering, briefly: problem and scope; agent design (topology, tools, verification, loop limits); an architecture diagram with deployment boundaries marked; design decisions and what was rejected; evaluation results before and after the change; the p95 target, measured results, bottleneck and cost per 100 runs; and what did not work.

Repository — the README must be sufficient for someone unfamiliar with the project to run the system without assistance. This will be tested. Include the runbook, incident log, `.env.example` and evaluation set.

12. Cost

All software and data required are free. The corpus documents are public; **sentence-transformers run locally for embeddings at no cost**, and a small model such as all-MiniLM-L6-v2 keeps memory use low; FAISS, Chroma, LangGraph, CrewAI, LangChain, LlamaIndex, TruLens, FastAPI, Redis, Prometheus, Grafana, OpenTelemetry, Jaeger and Locust are all free. For the model, use a provider's free tier. A local model via Ollama is also possible but needs a considerably more powerful machine.

Embed locally. Embedding a corpus through a paid API is the only operation here with a real cost.

Agents multiply model calls — typically several per run, against one for a fixed pipeline. Free tiers also have low rate limits, which is why load testing uses a stub (D6). This is the reason for the loop limits in A1, the caching in D7 and the per-step cost tracking in L5. Set a spending limit on the account before starting, and place the model behind an interface you control so the provider can be changed by configuration.

13. Out of Scope

- Model training is not required.
- Cloud deployment is not required; the system runs locally under Docker Compose.
- **Use synthetic case reports only.** No real patient or case data may enter the system at any point.